

Class 6: R Functions

Jolie Adair (PID A17337497)

Table of contents

Background	1
Our first functions	1
A second function	2
A Protein generating function	5

Background

ALL FUNCTIONS IN R HAVE AT LEAST 3 THINGS:

- A **name** that we use to call the cfunction.
- One or more input **arguments**
- The **body** the lines of R code that do the work

Our first functions

Let's write a silly wee function called `add()` to add some numbers (the input arguments)

```
add <- function(x, y) {  
  x + y  
}
```

Now we can use this function

```
add(x=100, y=1)
```

```
[1] 101
```

```
add(x=c(100, 1, 100), y=1)
```

```
[1] 101    2 101
```

Q. What if I give a multiple element vector to **x** and **y**

```
add(c(100, 1), y=c(100,1))
```

```
[1] 200    2
```

Q. What if I give three inputs to the functions?

```
#add(x=c(100,1), y=1, z=1 )
```

Q. What if I give only one input to the add function?

```
addnew <- function(x, y=1) {  
  x + y  
}
```

```
addnew(x=100)
```

```
[1] 101
```

```
addnew(c(100,1), 100)
```

```
[1] 200 101
```

If we write out functions with input arguments having no default value then the user will be required to set them when they use the function. We can give our input argument “default” values by setting them equal to some sensible value - e.g. `y=1` in the `add()`

A second function

Let's try something more interesting: Make a sequence generating tool..

The `sample()` function can be a useful starting point here:

```
sample(1:10, size=4)
```

```
[1] 6 2 10 8
```

Q. Generate 9 random numbers taken from the input vector x=1:10?

```
sample(1:10, size=9)
```

```
[1] 4 7 8 1 3 2 9 6 10
```

Q. Generate 12 random numbers taken from the input vector x=1:10?

```
sample(1:10, size=12, replace = TRUE)
```

```
[1] 4 9 6 10 4 6 9 2 1 9 9 6
```

Q. Write code for the `sample()` functions that generates nucleotide sequences of length 6?

```
sample(x=c("A", "G", "T", "C") , size =6, replace = TRUE, )
```

```
[1] "T" "T" "A" "C" "C" "T"
```

Q. Write a first function `generate_dna()` that returns a **user specified length** DNA sequence:

```
generate_dna <- function(len=6) {  
  sample(x=c("A", "G", "T", "C") , size =len, replace = TRUE, )  
}
```

```
generate_dna(len = 100)
```

```
[1] "A" "T" "C" "G" "T" "G" "C" "T" "G" "G" "A" "T" "T" "T" "G" "G" "G" "G"  
[19] "C" "A" "C" "T" "A" "G" "A" "T" "G" "G" "C" "G" "A" "A" "T" "G" "A" "A"  
[37] "A" "T" "C" "G" "C" "G" "A" "A" "C" "G" "G" "G" "C" "A" "A" "A" "T" "A"  
[55] "C" "T" "G" "T" "C" "T" "C" "A" "C" "C" "C" "G" "T" "A" "T" "G" "G" "T"  
[73] "C" "A" "G" "G" "T" "G" "C" "C" "C" "T" "C" "A" "A" "A" "C" "G" "T" "T"  
[91] "C" "C" "T" "A" "A" "A" "G" "G" "C" "A"
```

Key Points

Every function in R looks fundamentally the same in the terms of its structure. Basically 3 things: name, input, and body

```
name <- function(input) {  
  body  
}
```

Functions can have multiple inputs. These can be **required** arguments or **optional** arguments. With optional arguments having a set value.

Q. Modify and improve our `generate_dna()` function to return it's generated sequence in a more standar format like "AGTAGTA" rather than the vector "A", "G", "T", "C"

```
generate_dna <- function(len=6, fasta =TRUE) {  
  
  ans <- sample(x=c("A", "G", "T", "C") ,  
               size =len,  
               replace = TRUE, )  
  if(fasta) {  
    cat("Single element vector output")  
    ans <- paste(ans, collapse = "")  
  } else {  
    cat("Multi-element vector output")  
  }  
  return(ans)  
}  
  
generate_dna(fasta=FALSE)
```

Multi-element vector output

```
[1] "G" "A" "G" "T" "A" "A"
```

```
generate_dna(fasta=TRUE)
```

Single element vector output

```
[1] "CTCCAT"
```

The `paste()` function - it's job is to join up or stick together(a.k.a. paste) input strings together

```
paste("diego", "loves R", sep=" does not ")
```

```
[1] "diego does not loves R"
```

Flow control means where the R brain goes in your code

```
good_mood <- FALSE

if(good_mood) {
  cat("Great!")
} else {
  cat("Bummer!")
}
```

Bummer!

A Protein generating function

- Q. Write a function that generates a user specified length protein sequence.
- Q. Use that function to generate random protein sequences between length 6 to 12.
- Q. Are any of your sequences unique i.e. not found anywhere in nature?

```
aa <- c("A", "R", "D", "C", "Q", "E", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T", "W")
```

```
generate_protein <- function(len) {
  # The amino-acids to sample from

  aa <- c("A", "R", "D", "C", "Q", "E", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T", "W")
  #Draw n=len amino acids to make our sequence

  ans <- sample(aa, size = len, replace = T)
  ans <- paste(ans, collapse= "")
  return(ans)
}
```

```
myseq <- generate_protein(42)
myseq
```

```
[1] "SQNSGEMQHEKMIEGEDALKSYFWIRAVHCTCFSSPPEPRFN"
```

Q.2 answer

```
generate_protein(6)
```

```
[1] "ISFDLE"
```

```
generate_protein(7)
```

```
[1] "FLLDARF"
```

Q. 3 answer

```
for(i in 6:12) {
  # FASTA ID line ">id"
  cat(">", i, sep="", "\n")
  # Protein Sequence
  cat( generate_protein(i), "\n" )
}
```

```
>6
SMIWIY
>7
WGKDDHV
>8
QNRVARIR
>9
VWPTVWQGV
>10
VLGGGYTFSV
>11
YRCWKRGRFTT
>12
PMKKFGIFVIMA
```